

Documentação do Simplex

Naan Vasconcelos e Thiago Braga

June 27, 2025

Sumário

1	Introdução	3
1.1	Contexto e Motivação	3
1.2	Objetivos do Projeto	3
1.3	Escopo e Limitações	3
2	Arquitetura do Simplex	4
3	Módulos e Interfaces	4
3.1	linear_program.py (Passo 1)	4
3.2	tableau.py (Passo 2 e 3)	4
3.3	solver.py (Passo 4)	5
3.4	main.py (Passo 5)	5
4	Estratégias de Resolução	5
5	Testes de Execução	5
6	Conclusão	5

1 Introdução

1.1 Contexto e Motivação

Em cenários de otimização do mundo real—como roteirização de veículos, planejamento de produção, alocação de recursos financeiros e gestão de cadeias de suprimentos—organizações enfrentam problemas complexos em que é necessário maximizar lucros ou eficiência, sujeito a múltiplas restrições operacionais. A Programação Linear (PPL) é a abordagem padrão para modelar esse tipo de desafio, e o Método Simplex, criado por George Dantzig em 1947, permanece até hoje o algoritmo clássico e mais utilizado para encontrar soluções ótimas, navegando de maneira sistemática pelos vértices extremos da região viável.

Este projeto apresenta uma implementação completa do Método Simplex em Python, incluindo:

- Leitura de modelos de PPL a partir de arquivos de texto;
- Tratamento de restrições $\leq$, $\geq$ e $=$, com introdução de variáveis de folga, excesso e artificiais;
- Implementação das Fases I e II, garantindo a obtenção de uma base viável e a otimização do problema;
- Identificação de múltiplas soluções básicas ótimas adjacentes;
- Geração de relatórios formatados com `pandas` para facilitar a análise dos resultados.

Limitações: não há suporte nativo para problemas de minimização (embora possam ser convertidos), variáveis livres ou casos em que o vetor b contenha valores negativos.

1.2 Objetivos do Projeto

- **Principal:** implementar do zero todas as fases do método Simplex em Python, sem uso de solvers prontos.
- **Secundários:**
 - Encontrar bases iniciais via Fase I.
 - Detectar soluções ilimitadas e múltiplas ótimas.
 - Gerar relatórios formatados com `pandas`.

1.3 Escopo e Limitações

- Problemas de **maximização** com variáveis não-negativas e vetor $b \geq 0$.
- Não há suporte a variáveis livres, minimização direta (embora possa ser convertida) nem $b < 0$.

2 Arquitetura do Simplex

Este método resolve problemas de Programação Linear, cujo modelo canônico é:

$$\max \mathbf{c}^T \mathbf{x} \text{ sujeito a } A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq 0,$$

onde:

- $\mathbf{c} \in R^n$ é o vetor de coeficientes da função objetivo;
- $A \in R^{m \times n}$ é a matriz de coeficientes das restrições;
- $\mathbf{b} \in R^m$ é o vetor de termos independentes (lado direito).

Este diagrama textual apresenta o percurso principal do Simplex, do momento em que o usuário fornece um modelo até a entrega dos resultados:

1. **Carregamento e validação do modelo** (Passo 1): recebe vetores $\mathbf{c}$, $\mathbf{b}$ e matriz A , valida dimensões e sinais.
2. **Construção do tableau inicial** (Passo 2): monta o tableau padrão a partir do modelo validado.
3. **Fase I – Solução viável** (Passo 3): executa iterações até encontrar ou comprovar inexistência de solução viável.
4. **Fase II – Otimização** (Passo 4): aplica iterações de otimização sobre a base viável.
5. **Formatação e saída dos resultados** (Passo 5): extrai a solução básica ótima e gera relatório final.

3 Módulos e Interfaces

Cada componente a seguir implementa um dos passos descritos na Seção 2.

3.1 `linear_program.py` (Passo 1)

Encapsula o modelo de Programação Linear (PPL): armazena os vetores $\mathbf{c}$, $\mathbf{b}$ e a matriz A , além dos sinais das restrições. Valida dimensões e oferece um construtor alternativo (`from_file`) para carregar diretamente de um arquivo de texto.

3.2 `tableau.py` (Passo 2 e 3)

Implementa a lógica do método Simplex em duas fases. A classe `SimplexTableau` recebe um `LinearProgram`, monta o tableau inicial (incluindo variáveis de folga e artificiais), percorre os pivôs para alcançar a solução ótima na Fase I e depois na Fase II, e extrai tanto a solução ótima quanto eventuais soluções alternativas adjacentes.

3.3 solver.py (Passo 4)

Fornece a interface simplificada: a classe `SimplexSolver` lê o arquivo de entrada, inicializa o tableau e dispara o processo de otimização. Seu único método público, `solve()`, retorna o triplo (solução x , valor ótimo z , lista de alternativas).

3.4 main.py (Passo 5)

Define o ponto de entrada da aplicação CLI. Recebe o nome do arquivo pela linha de comando, invoca o `SimplexSolver`, e utiliza o `pandas` para formatar e exibir no console o valor ótimo, o vetor solução e quaisquer soluções ótimas adjacentes.

4 Estratégias de Resolução

- **Fase I:** minimiza artificiais;
- **Fase II:** maximiza a função original;
- **Desempates:** arbitrários (primeira negativa / menor razão);
- **Múltiplas ótimas:** pivôs com custo reduzido zero.

5 Testes de Execução

Exemplo 1 — Solução única

- Modelo: $\max 3x_1 + 5x_2$, restrições: $x_1 \leq 4$, $2x_2 \leq 12$, $3x_1 + 2x_2 \leq 18$;
- Arquivo: `entrada_unica.txt`;
- Saída: $x_1 = 2$, $x_2 = 6$, $z = 36$.

Exemplo 2 — Múltiplas soluções

- Modelo: $\max x_1 + x_2 + x_3$, com $x_1 + x_2 + x_3 \leq 1$;
- Arquivo: `entrada_multiplas.txt`;
- Saída: $x_1 = 1$ (principal), $x_2 = 1$ e $x_3 = 1$ (alternativas).

6 Conclusão

A implementação cobre leitura flexível de PPL, fases I e II, desempates e enumeração de múltiplas soluções básicas. É facilmente extensível para novos formatos, minimização ou interfaces gráficas.